

Credit Card Approval Prediction System

Project Description:

The Credit Card Approval Prediction System is a machine learning-based application designed to automate the credit card approval process in banking and financial institutions. The system analyzes applicant financial and demographic information such as income type, employment duration, education level, housing type, family status, and credit history to predict whether a credit card application should be approved or rejected.

The project utilizes multiple machine learning classification algorithms including Logistic Regression, Decision Tree, Random Forest, and XG Boost to build predictive models. After evaluating model performance using accuracy, confusion matrix, and classification reports, the best-performing model is selected and deployed.

The trained model is integrated into a Flask web application that allows users to enter applicant details and receive instant approval predictions through a user-friendly interface. The project also supports IBM Watson Machine Learning deployment for scalable cloud-based predictions.

Scenarios

Scenario 1: Automated Credit Card Application Screening

A bank credit analyst enters an applicant's financial information including income type, employment duration, family status, and credit history into the application. The system instantly predicts whether the application is likely to be approved or rejected.

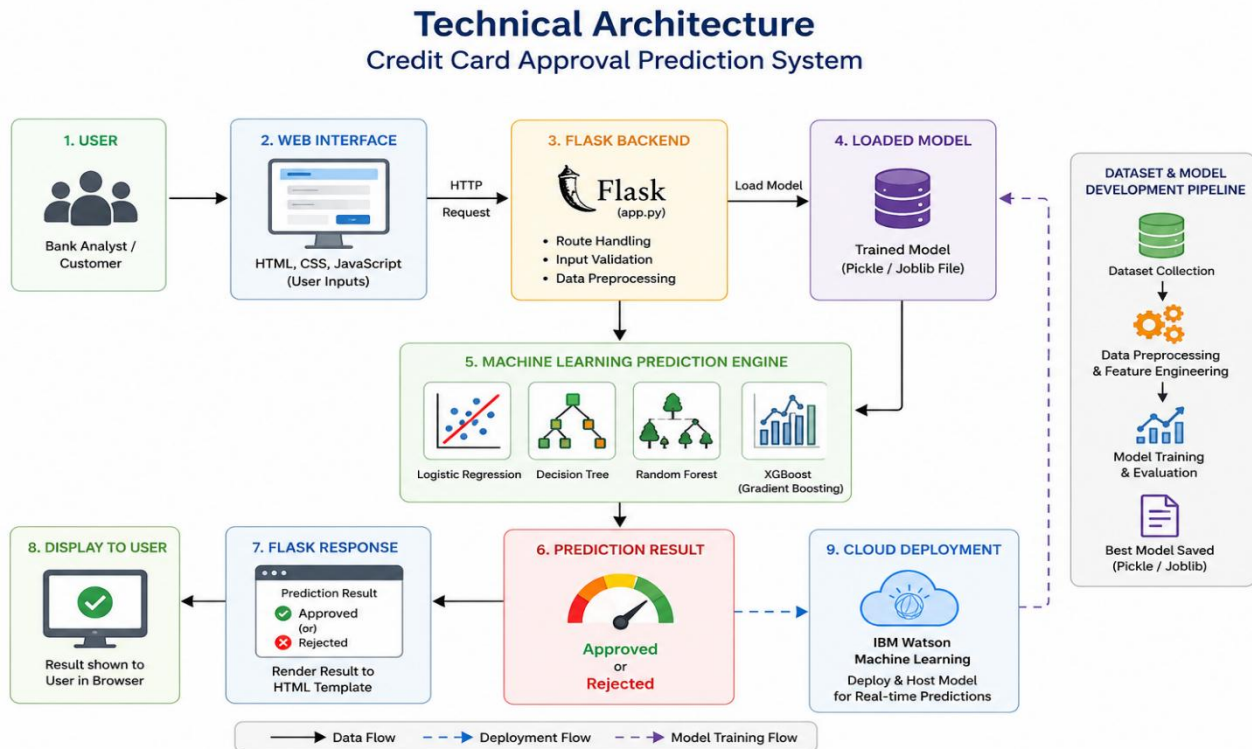
Scenario 2: High-Risk Applicant Identification

A compliance officer evaluates applicants with poor repayment records and overdue payment histories. The system identifies high-risk applicants and helps reduce financial risk by recommending rejection when necessary.

Scenario 3: Customer Self-Service Eligibility Check

A customer enters personal and financial information into the web application before applying for a credit card. The system predicts the likelihood of approval, helping customers understand their eligibility before submitting a formal application.

Technical Architecture:



Description: The Credit Card Approval Prediction System follows a three-tier architecture consisting of the user interface, Flask backend, and machine learning model. Users enter applicant details through the web application, which are validated and processed by the Flask server. The processed data is then passed to the trained machine learning model to predict whether the credit card application will be approved or rejected. Finally, the prediction result is displayed to the user and can also be deployed on IBM Watson Machine Learning for real-time cloud-based predictions.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Groq API Familiarity:** [Groq API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Ngrok for Public Deployment:** [Ngrok Documentation](#)

Project Workflow:

Milestone 1: Environment Setup and Project Planning

- **Activity 1.1:** Install Python and configure the development environment.
- **Activity 1.2:** Install the required libraries (NumPy, Pandas, Scikit-learn, Flask).
- **Activity 1.3:** Set up Git and GitHub for version control.
- **Activity 1.4:** Create the project structure and organize project files.

Milestone 2: Data Collection and Preprocessing

- **Activity 2.1:** Load the credit card approval dataset using Pandas.
- **Activity 2.2:** Analyze the dataset and understand its features.
- **Activity 2.3:** Handle missing values and remove duplicate records.
- **Activity 2.4:** Perform data preprocessing and prepare the dataset for model training.

Milestone 3: Model Development

- **Activity 3.1:** Split the dataset into training and testing datasets.
- **Activity 3.2:** Train the machine learning model using Scikit-learn.
- **Activity 3.3:** Evaluate the model performance and save the trained model.

Milestone 4: Web Application Development

- **Activity 4.1:** Design the user interface using HTML and CSS.
- **Activity 4.2:** Develop the Flask backend to handle user requests.
- **Activity 4.3:** Integrate the trained machine learning model with the Flask application.
- **Activity 4.4:** Display the prediction results through the web interface.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the application locally and verify prediction accuracy.
- **Activity 5.2:** Upload the project to GitHub for version control and collaboration.
- **Activity 5.3:** Deploy the Flask application and verify end-to-end functionality.

Milestone 6: Documentation and Conclusion

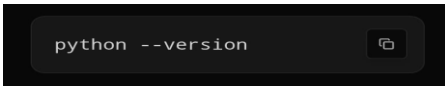
- **Activity 6.1:** Prepare the project documentation with architecture, workflow, and screenshots.
- **Activity 6.2:** Summarize the project results and identify possible future improvements.

Milestone 1: Environment Setup and Project Planning

In this milestone, the development environment is prepared, and the project structure is organized. The required software, libraries, and tools are installed to ensure smooth implementation of the Credit Card Approval Prediction System.

Activity 1.1: Install Python and Development Tools

- Install Python 3.8 or above.
- Install Anaconda Navigator or Visual Studio Code.
- Verify Python installation and configure the development environment.



```
python --version
```

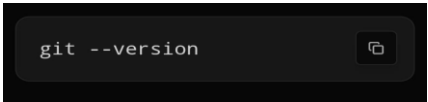
Activity 1.2: Install Required Libraries

- Install the required Python libraries such as NumPy, Pandas, Scikit-learn, Flask, Matplotlib, and Seaborn.
- Verify that all dependencies are installed successfully.

~pip install numpy pandas matplotlib seaborn scikit-learn xg boost joblib flask

Activity 1.3: Configure Git and GitHub

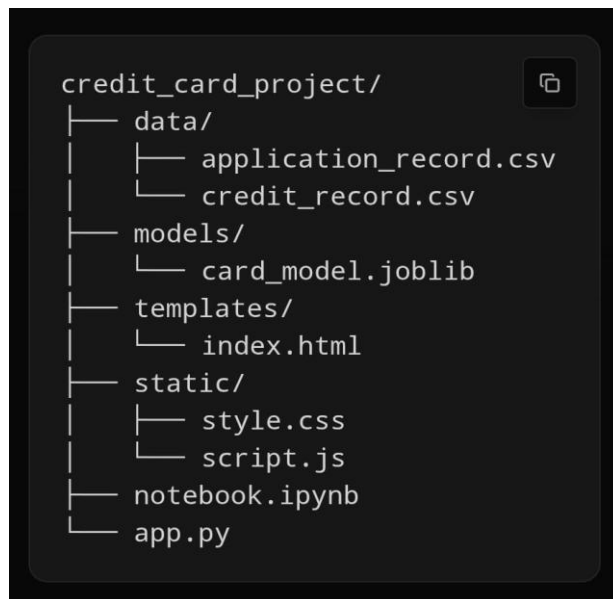
- Install Git on the system.
- Create a GitHub repository for the project.
- Initialize the local repository and connect it with GitHub.



```
git --version
```

Activity 1.4: Organize Project Structure

- Create folders for datasets, notebooks, templates, static files, and trained models.
- Organize the project files for easy development and maintenance.



Milestone 2: Data Collection and Preprocessing

This milestone focuses on preparing the dataset for machine learning by performing data analysis, cleaning, and preprocessing.

Activity 2.1: Load and Explore the Dataset

- Import the dataset using Pandas.
- Understand the dataset structure, features, and target variable.
- Display sample records and dataset information.

Activity 2.2: Analyze the Dataset

- Perform exploratory data analysis (EDA).
- Generate graphs and visualizations using Matplotlib and Seaborn.
- Identify important features affecting credit card approval.

Activity 2.3: Clean the Dataset

- Remove duplicate records.
- Handle missing values.
- Convert categorical values into numerical format.

Activity 2.4: Prepare Data for Training

- Perform feature engineering if required.
- Split the dataset into training and testing datasets using Scikit-learn.

Milestone 3: Model Development

In this milestone, the machine learning model is trained, evaluated, and saved for integration into the web application.

Activity 3.1: Train the Machine Learning Model

- Import the required Scikit-learn modules.
- Train the model using the prepared dataset.
- Optimize the model for better prediction performance.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib
import os

def run_training_pipeline():
    print("Step 1: Loading raw datasets...")
    app_df = pd.read_csv('data/application_record.csv')
    credit_df = pd.read_csv('data/credit_record.csv')

    print("Step 2: Constructing target variable and merging records...")
    # Label status '2', '3', '4', or '5' as risky (1)
    credit_df['is_risky'] = credit_df['STATUS'].isin(['2', '3', '4', '5']).astype(int)

    # Identify if a client has defaulted at least once
    user_target = credit_df.groupby('ID')['is_risky'].max().reset_index()
    user_target.rename(columns={'is_risky': 'target'}, inplace=True)

    # Merge datasets
    df = pd.merge(app_df, user_target, on='ID', how='inner')

    print("Step 3: Preprocessing and resolving anomalies...")
    # Replace null occupation entries
    df['OCCUPATION_TYPE'] = df['OCCUPATION_TYPE'].fillna('Unknown')

    # Convert birth and employment day offsets to years
    df['AGE'] = (df['DAYS_BIRTH'] / -365.25).astype(int)
    df['YEARS_EMPLOYED'] = df['DAYS_EMPLOYED'].apply(lambda x: 0 if x > 0 else x / -365.25)
    df['UNEMPLOYED'] = (df['DAYS_EMPLOYED'] > 0).astype(int)

    # Drop raw and redundant ID/offset variables
    df_clean = df.drop(columns=['ID', 'DAYS_BIRTH', 'DAYS_EMPLOYED', 'FLAG_MOBIL'])

    print("Step 4: Categorical Label Encoding...")
    le = LabelEncoder()
    cat_cols = ['CODE_GENDER', 'FLAG_OWN_CAR', 'FLAG_OWN_REALTY', 'NAME_INCOME_TYPE',
                'NAME_EDUCATION_TYPE', 'NAME_FAMILY_STATUS', 'NAME_HOUSING_TYPE', 'OCCUPATION']
    for col in cat_cols:
        df_clean[col] = le.fit_transform(df_clean[col].astype(str))

    # Separate features and target
    X = df_clean.drop(columns=['target'])
    y = df_clean['target']

    # Stratified split to keep target ratio balanced
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    print("Step 5: Training and evaluating classifiers...")
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, class_weight='balanced'),
        "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced'),
        "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss')
    }

    best_acc = 0.0
    best_model = None
    best_model_name = ""

    for name, clf in models.items():
        # Fit classifier
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        print(f"> {name} Accuracy: {acc * 100:.2f}%")

        # Track the highest performing model
        if acc > best_acc:
            best_acc = acc
            best_model = clf
            best_model_name = name
```

Activity 3.2: Evaluate Model Performance

- Test the model using the testing dataset.
- Measure prediction accuracy.
- Analyze the model's performance using evaluation metrics.

Activity 3.3: Save the Trained Model

- Save the trained model using Pickle or Joblib.
- Verify that the model can be loaded successfully for prediction.

Milestone 4: Web Application Development

This milestone focuses on developing a user-friendly web application using Flask, HTML, and CSS.

Activity 4.1: Design the User Interface

- Create responsive web pages using HTML and CSS.
- Design forms to collect applicant information.
- Ensure the interface is simple and easy to use.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Credit Card Approval Predictor</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <div class="container">
    <h2>Credit Card Eligibility Predictor</h2>
    <form id="predictionForm">
      <label for="gender">Gender:</label>
      <select id="gender" required>
        <option value="0">Female</option>
        <option value="1">Male</option>
      </select>

      <label for="own_car">Owns a Car?</label>
      <select id="own_car" required>
        <option value="0">No</option>
        <option value="1">Yes</option>
      </select>

      <label for="own_realty">Owns Real Estate / Property?</label>
      <select id="own_realty" required>
        <option value="0">No</option>
        <option value="1">Yes</option>
      </select>

      <label for="children">Number of Children:</label>
      <input type="number" id="children" min="0" value="0" required>

      <label for="income">Annual Income ($):</label>
      <input type="number" id="income" min="0" required>

      <label for="income_type">Income Type:</label>
      <select id="income_type" required>
        <option value="4">Working</option>
        <option value="0">Commercial associate</option>
        <option value="1">Pensioner</option>
        <option value="2">State servant</option>
        <option value="3">Student</option>
      </select>

      <label for="education">Highest Education Level:</label>
      <select id="education" required>
        <option value="4">Secondary / secondary special</option>
        <option value="1">Higher education</option>
        <option value="2">Incomplete higher</option>
        <option value="3">Lower secondary</option>
        <option value="0">Academic degree</option>
      </select>

      <label for="family_status">Family Status:</label>
      <select id="family_status" required>
        <option value="1">Married</option>
        <option value="3">Single / not married</option>
        <option value="0">Civil marriages</option>
        <option value="2">Separated</option>
        <option value="4">Widow</option>
      </select>

      <label for="housing_type">Housing Type:</label>
      <select id="housing_type" required>
        <option value="1">House / apartment</option>
        <option value="5">With parents</option>
        <option value="2">Municipal apartment</option>
        <option value="4">Rented apartment</option>
        <option value="3">Office apartment</option>
        <option value="0">Co-op apartment</option>
      </select>

      <label for="occupation">Occupation Type:</label>
      <select id="occupation" required>
        <option value="18">Unknown</option>
        <option value="8">Laborers</option>
        <option value="3">Core staff</option>
        <option value="14">Sales staff</option>
        <option value="10">Managers</option>
      </select>
    </form>
  </div>
</body>
</html>
```

Activity 4.2: Develop the Flask Backend

- Create the Flask application.
- Define routes for the home page and prediction page.
- Handle user input and validate form data.

```
from flask import Flask, request, jsonify, render_template
import joblib
import numpy as np

app = Flask(__name__)

# Load the trained model
model = joblib.load('models/card_model.joblib')

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.get_json()

        # Build features list in the exact order the model expects:
        # CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, CNT_CHILDREN, AMT_INCOME_TOTAL,
        # NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE,
        # OCCUPATION_TYPE, CNT_FAM_MEMBERS, AGE, YEARS_EMPLOYED, UNEMPLOYED
        features = np.array([
            int(data['gender']),
            int(data['own_car']),
            int(data['own_realty']),
            int(data['children']),
            float(data['income']),
            int(data['income_type']),
            int(data['education']),
            int(data['family_status']),
            int(data['housing_type']),
            int(data['occupation']),
            int(data['family_size']),
            int(data['age']),
            float(data['years_employed']),
            int(data['unemployed'])
        ])

        # Predict target (0 = Safe/Approved, 1 = Risky/Rejected)
        prediction = model.predict(features)[0]

        # Invert label: Approved is true if prediction is 0
        approved = int(prediction) == 0

        return jsonify({'approved': approved})
    except Exception as e:
        return jsonify({'error': str(e)}), 400

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Activity 4.3: Integrate the Machine Learning Model

- Load the saved machine learning model.
- Pass user input to the model.
- Generate approval or rejection predictions.

Activity 4.4: Display Prediction Results

- Show the prediction result on the webpage.
- Display whether the credit card application is Approved or Rejected.
- Handle invalid input and error messages.

Milestone 5: Testing and Deployment

This milestone ensures that the application functions correctly and is ready for deployment.

Activity 5.1: Perform Application Testing

- Test the application using different applicant details.
- Verify the accuracy of prediction results.
- Fix bugs and improve performance.



**Activity 5.2: Version Control Using GitHub**

- Upload the complete project to GitHub.
- Maintain version history and manage project updates.

Activity 5.3: Deploy the Application

- Deploy the Flask application.
- Verify that the application works correctly after deployment.
- Ensure users can access the prediction system through the web interface.

Milestone 6: Documentation and Conclusion

This milestone focuses on preparing the final project documentation and summarizing the project outcomes.

Activity 6.1: Prepare Project Documentation

- Document the system architecture, workflow, implementation steps, and results.
- Include screenshots, diagrams, and project outputs.

Activity 6.2: Summarize Project Outcomes

- Review the overall performance of the Credit Card Approval Prediction System.
- Highlight the project's benefits, limitations, and future enhancement opportunities.

Conclusion:

The **Credit Card Approval Prediction System** successfully demonstrates how machine learning can be used to automate the credit card approval process by analyzing applicant information and predicting whether an application is likely to be approved or rejected. The project was developed using **Python, NumPy, Pandas, Scikit-learn, Flask, HTML, CSS, Git, and GitHub**, providing a simple and efficient web-based solution for real-time predictions.

Throughout the project, data preprocessing, model development, and web application integration were completed to ensure accurate and reliable predictions. The Flask-based application offers an easy-to-use interface for users to enter applicant details and instantly receive prediction results. Overall, the project provides practical experience in machine learning, data preprocessing, web development, and version control while demonstrating the application of AI in the banking and financial sector. Future enhancements may include improving model accuracy with larger datasets, adding user authentication, integrating cloud deployment, and incorporating advanced machine learning techniques for enhanced performance.